GIDFINE REPOSITORY ALIGNMENT & CONVEX ROADMAP AUDIT

Reference: **Gidfine_Dev_Guide.md (v1.1)** | Target Stack: React 19 + TypeScript + Convex | Branch: development

1. Executive Overview & Repository Baseline

This audit examines the current alignment of the development branch against Matthias's approved product and architectural specification (**React + TypeScript + Convex**) defined in *Gidfine_Dev_Guide.md*. The repository is cleanly unified under React 19 with all legacy Vue components completely excised. However, Convex has not yet been initialized, and the application remains coupled to legacy Python/FastAPI backend components. This audit establishes the reusable frontend components, identifies architectural gaps, and presents a phased mini-scale plan.

2. Frontend Inventory & Reusability Assessment

Component / Domain	Source Location	Dev Guide Role / Feature	Reusability
Authentication	components/Auth.tsx, Login.tsx, AuthModal.tsx	FR-01: User registration & role credentials	High (adapt to Convex Auth)
Workspace Layout	workspace/layout/DashboardLayout.tsx	Sec 10: Multi-portal shell & navigation	High (add role routing)
Origination	workspace/pages/LoanApplication.tsx	FR-02: Borrower application capture	High (switch to Ghana Card)
Calculator & Sim	components/WhatIfSimulator.tsx, EmiDonutChart.tsx	FR-06: Schedule & margin preview	High (integer minor units)
Matching & Tables	components/LoanComparisonTable.tsx, Pane.tsx	FR-05: Lender matching & quote cards	High (field masking required)
Agency Dashboard	components/AdminDashboard.tsx, Analytics.tsx	FR-13: Review queue & metrics	Medium (wire to live queries)

3. Specification Gap Analysis vs Gidfine_Dev_Guide.md

Dimension	Dev Guide Requirement	Current Repository State	Severity
Data & Runtime	Convex reactive DB & backend functions	FastAPI + local MongoDB (convex uninstalled)	High
Tenancy Model	Strict tenant-level isolation (agencyId on tables)	Single-tenant global database collections	High
Role Controls	7 distinct roles (Agency Admin, Officer, Lender, etc.)	Binary customer vs admin role checks	High
Disclosure Masking	Server-side field redaction (FR-08: no phone to lenders)	Open JSON payloads without server-side masking	Critical
Financial Math	Integer minor units (pesewas), immutable ledger	Floating-point binary calculation formulas	Critical
State Machine	11-state transition workflow with dual approval	Linear client-side wizard states	High

4. Recommended Mini-Scale End-to-End Implementation Slice

To validate the target architecture without over-engineering, the initial Convex implementation should model a complete single-loan transaction lifecycle: **One Agency, One Borrower, One Lender**:

- **Step 1 (Origination)**: Borrower submits loan request (e.g., GHS 5,000 for 6 months) via React UI.
- **Step 2 (Agency Pricing)**: Agency sets lender base rate (18% p.a.) and agency margin (300 bps / 3%) in pesewas.
- **Step 3 (Lender Review)**: Lender views redacted proposal (financial summary only; borrower identity masked) and accepts.
- **Step 4 (Agency Approval & Borrower Consent)**: Agency validates offer; borrower affirmatively reviews and signs consent.
- **Step 5 (Disbursement & Servicing)**: Disbursement is confirmed; 1st repayment posting updates an append-only ledger in Convex.

5. Critical Decisions Requiring Matthias's Approval

1. **Tenancy Structure**: Should the initial Convex schema enforce multi-tenant SaaS (agencyId) or single agency?
2. **Calculation Standard**: Approved formula for Ghana market (flat-rate simple interest vs reducing-balance amortizing)?
3. **Margin Mechanism**: Percentage-point spread (added to rate) vs fixed upfront origination fee?
4. **Lender Disclosures**: Exact legal requirement for borrower visibility on lender-facing screens under Act 843?
5. **Payment Settlement**: Manual entry by Cashier for MVP vs live integration with Ghana Mobile Money (MoMo) rails?